SEG 4180 – Lab 2
Siva Senthilkumaran (Student #: 300230364)

This project builds on the pipeline developed in Lab 1, replacing the pretrained DistilBERT sentiment classifier with a custom-trained computer vision model for semantic segmentation of buildings in aerial imagery. The project also introduces secrets injection, automated testing, and a CI/CD pipeline that builds and pushes Docker images directly to Docker Hub.

Semantic segmentation assigns a class label to every pixel in an image, unlike object detection (that uses bounding boxes) and instance segmentation (that applies per object). The task for this project is a binary semantic segmentation problem: each pixel is either classified as a “building” or “background”.

The complete pipeline covers dataset acquisition (from Kaggle), pixel mask generation, patch-based tiling, model training with BCE and Dice loss, quantitative evaluation with IoU and Dice scores, a REST API for serving and CI/CD with GitHub Actions.

The dataset used is “Semantic Segmentation of Aerial Imagery” published by Humans in the Loop on Kaggle. It consists of aerial satellite imagery of Dubai and UAE. The raw data contains large tiles with corresponding RGB segmentation masks spanning six classes: Building, Land, Road, Vegetation, Water and Unlabeled. The dataset is downloaded with the Kaggle API. Its slug and target class color are configurable with environment variables so that any Kaggle dataset can be used in the pipeline with appropriate configuration.

The raw masks encode all six classes as RGB colors. Hence, preprocessing is required to extract the building class into binary masks. Rgb_mask_to_binary in the dataset code performs exact matching against the configured building color, producing a single-channel mask where 255 indicates building pixels and 0 indicates background.

The large source tiles are sliced into non-overlapping 256x256 patches with a sliding window approach, yielding 1305 total patches from the source images. Patches are shuffled with a fixed random seed and split into 70% train, 15% validation and 15% test.

Training patches are augmented at load time using Albumentations. Random flips (both horizontal and vertical) and 90 degree rotations are applied along with brightness and contrast adjustments. These augmentations are applied to both the image and mask, but validation and test sets are only resized and normalized.

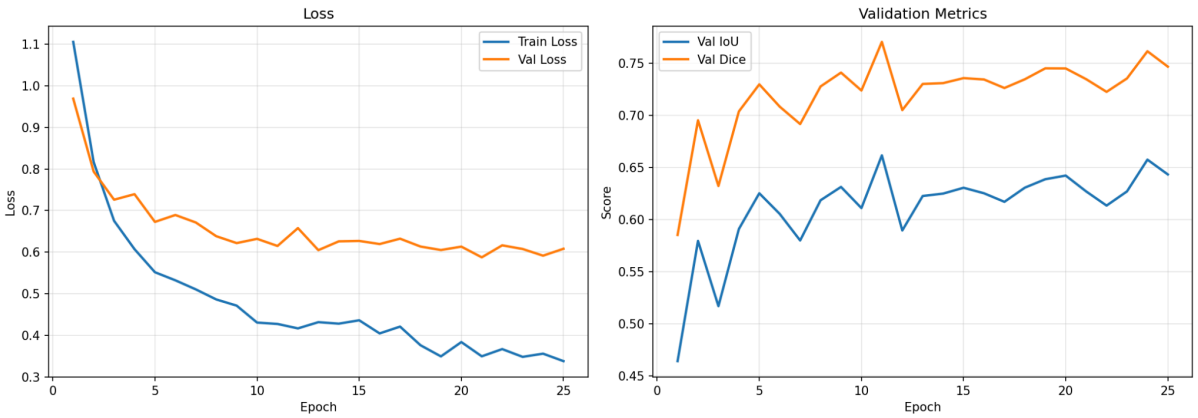
The model used is a U-Net with a ResNet34 encoder, implemented through segmentation-models-pytorch. The encoder is initialized with ImageNet pretrained weights for strong low-level feature representations that transfer well to aerial imagery. The decoder reconstructs spatial resolution through transposed convolutions and skip connections from corresponding encoder stages. The output is a single-channel logit map at the input resolution with no final activation.

The loss function combines binary cross-entropy with logits (BCE) and Dice loss. BCE provides stable gradients per pixel while Dice loss optimizes the overlap metric and

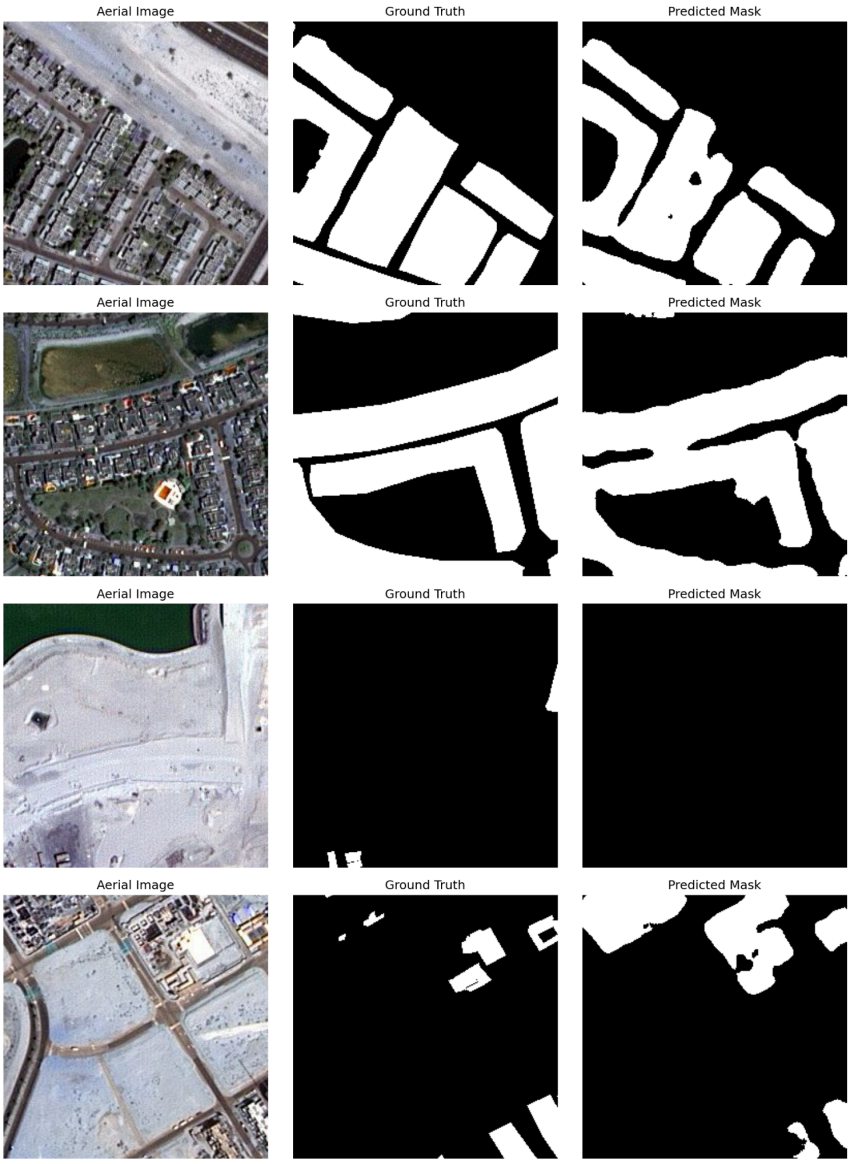
addresses class imbalances between building and ground pixels. The combined loss is the sum of both.

Training loss steadily decreased from 0.9709 to 0.3290 from epoch 0 to 25, which indicates the model was constantly learning. The best validation loss was 0.5792, achieved at epoch 18. At that point, the model was checkpointed. Overall, the model has a best validation IoU of 0.663 and a best validation Dice of 0.7707, which suggests reasonable performance for a modest dataset. The gap between training loss and validation loss suggests some overfitting, which is expected for a smaller dataset.

Training uses the Adam optimizer with an initial learning rate of 0.0001. A ReduceLROnPlateau scheduler halves the learning rate if validation loss does not improve for 3 epochs. The model checkpoint with the lowest validation loss is saved. Training ran for 25 epochs with a batch size of 4 on an NVIDIA GeForce RTX 4080.



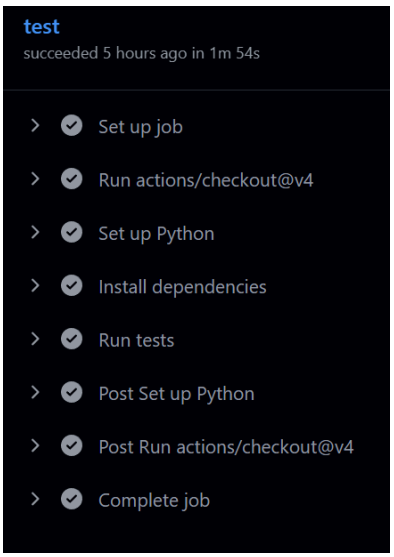
The decreasing training loss from 1.1060 to 0.3377 over epochs 0 to 25 show that the model was learning the training data, although the difference between training and validation loss suggests some overfitting, which is expected for a smaller dataset. The best validation loss achieved was 0.5875 at epoch 21, where the model was checkpointed. The test set's best achieved mean IoU was 0.6711 and the best achieved mean Dice was 0.7712. This is quite reasonable for a dataset of this size.

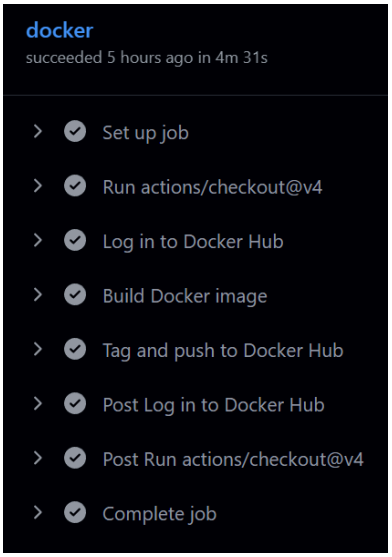


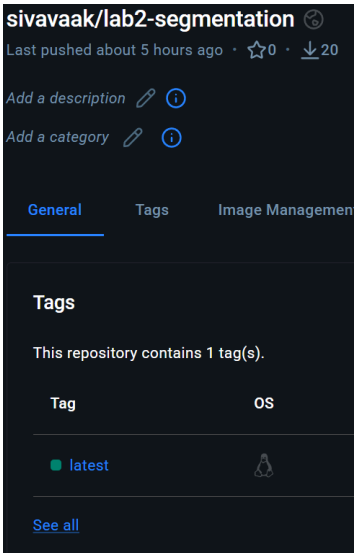
The samples shown demonstrate the model's ability to capture buildings with decent accuracy. It is able to recognize the major shapes of residential blocks, with distance representing roads and paths. However, in the third sample image, the predicted mask does not include the small parts of buildings in the corners of the patch. In this case, the combined BCE + Dice loss has likely learned that predicting all-black is close enough when buildings occupy such a small percentage of space in the patch. Additionally, residential buildings such as houses are blended together as large blocks, not accounting for backyards or sidewalks. However, this is a decision made in the creation of the dataset, as the annotators most likely filled the miniscule spaces representing gaps between houses. For the purposes of this project, the model can still be considered to be accurate. The fourth sample is the most inaccurate, as the model predicts two major blocks as buildings when they are labeled as backgrounds. However, these blocks could be mistaken for buildings by a human, so it may make sense for the model to predict incorrectly, even if they are actually storage crates or vehicles packed closely together. Aside from this, the model only has minor issues with rounded corners and blending separate blocks that are otherwise too close to each other, and performs reasonably in most cases.

Configuration and secrets such as Kaggle and Docker API keys are managed through environment variables. An example configuration is committed as a template with default values for all parameters: hyperparameters (batch size, learning rate, epochs), paths (data directory, checkpoint path, output directory), dataset configuration (Kaggle slug, building color RGB, patch size, split ratios), and service secrets (Flask secret key, Docker Hub credentials). The actual .env is never included via .gitignore. All configuration flows through a central config script that resolves relative paths against the root directory.

The GitHub Actions workflow runs on every push and pull request to the main branch. It runs both a test job and a Docker job. The test job sets up Python 3.11, installs all dependencies from requirements.txt and runs a specified test suite. This suite ensures the U-Net produces correct output shapes and that the sigmoid output falls between 0 and 1. It also verifies the IoU and Dice metric implementations against known cases: perfect overlap (1.0), no overlap (0.0), and partial overlap (scores in expected ranges). These tests run reliably in CI as they do not require any GPU or Dataset. The Docker job runs only on pushes to main and after the test succeeds. It builds a Docker image then pushes it to Docker Hub if a username and token are configured as repository secrets. The pipeline can still function in forks or environments where Docker Hub credentials cannot be provided.







The Dockerfile uses python3.11-slim as a base, installing dependencies, copying source code and running the Flask API through Waitress. The Flask API exposes the root endpoint (GET /) to return a health check, and the prediction endpoint (POST /predict), that accepts a multipart image upload, runs inference through the U-Net and returns a JSON response. This response contains the predicted mask as a base64-encoded PNG, the building coverage fraction (ratio of building to total pixels) and the input image dimensions.

Overall, this lab pipeline achieves all expectations with a reasonable level of fairness. The model demonstrates a performance high enough to argue the implementation of semantic segmentation with a pre-trained U-Net model does have merit. The biggest improvements to this project would be made with a larger or more detailed dataset. A bigger encoder or model fine-tuning may also yield better results.